

Week 3 Module Project A: Search and Adversarial Agents

Integrating pathfinding with minimax, alpha-beta pruning, and evaluation

CMPSC 480 - Instructor Solution Deck

Instructor material - do not distribute with the question deck

Solution overview

This instructor deck provides a complete matched solution and grading guidance.

Coverage: Weeks 1-3: search formulation, A-star, minimax, alpha-beta, and evaluation

Points

20 total

Expected time

10-14 hours

Submission

Repository, tests, and
engineering report

Instructor verification checklist

- ▶ Use Python 3.11 and NumPy only unless the prompt explicitly permits another package.
- ▶ Preserve the published interfaces and make all tie-breaking deterministic.
- ▶ State assumptions, validate inputs, and handle empty, terminal, and unreachable cases.
- ▶ Include focused tests whose names explain the defect each test would expose.
- ▶ Report measured evidence; do not replace experimental results with unsupported claims.
- ▶ Do not mutate game states shared by sibling branches.
- ▶ Return a legal action for every nonterminal state with at least one legal move.

Solution 1.a - Architecture and pathfinding (1 points)

Question focus. Draw the boundary among game state, search problem, pathfinder, and adversarial agent.

Answer. The state owns rules and legal transitions; the problem adapts a state to pathfinding; the pathfinder returns paths and metrics; the agent consumes immutable successor states and returns one action.

Solution 1.b - Architecture and pathfinding (2 points)

Question focus. Implement BFS, DFS, and A-star behind one pathfinding interface.

Answer. The methods share expansion and reconstruction logic, differ only in frontier priority, terminate on cycles, and return explicit failure on unreachable goals.

Solution 1.c - Architecture and pathfinding (1 points)

Question focus. Demonstrate path validity and deterministic tie-breaking.

Answer. Every adjacent state-action pair is revalidated by the domain. Equal priorities use a documented stable rule, and repeated runs produce identical paths.

Solution 2.a - A-star heuristic (1 points)

Question focus. Define the heuristic and its units.

Answer. A valid answer uses a lower bound expressed in path-cost units, such as obstacle-ignoring Manhattan distance times the minimum legal step cost.

Solution 2.b - A-star heuristic (1 points)

Question focus. Give an admissibility or consistency proof sketch.

Answer. Relaxing obstacles can only shorten the route, so the relaxed optimal cost is no greater than the true cost. For local consistency, show $h(n)$ no greater than edge cost plus h of each successor.

Solution 2.c - A-star heuristic (1 points)

Question focus. Compare A-star with $h=0$ on at least three maps.

Answer. Both configurations return equal optimal costs. The informative heuristic should usually expand no more nodes; exceptions caused by ties are explained.

Solution 3.a - Minimax decision engine (2 points)

Question focus. Write minimax pseudocode with MAX, MIN, terminal, and cutoff cases.

Answer. Terminal utility is returned before a heuristic cutoff. MAX takes the largest child value, MIN takes the smallest, and the root returns the action paired with the best value.

Solution 3.b - Minimax decision engine (2 points)

Question focus. Trace a depth-two tree with at least two actions per internal node.

Answer. The solution labels leaf utilities, computes each MIN value, then chooses the maximum at the root. All backed-up values and the selected action are shown.

Solution 3.c - Minimax decision engine (1 points)

Question focus. Explain how depth is counted and how ties are broken.

Answer. Depth is defined consistently as plies from the root or remaining plies. A stable legal-action order chooses the first tied optimum so tests remain reproducible.

Solution 4.a - Alpha-beta and evaluation (2 points)

Question focus. State the alpha and beta invariants and the pruning condition.

Answer. Alpha is MAX's best guaranteed lower bound; beta is MIN's best guaranteed upper bound. A branch can be pruned when alpha is at least beta because it cannot change the ancestor decision.

Solution 4.b - Alpha-beta and evaluation (1 points)

Question focus. Verify equality with plain minimax across generated small trees.

Answer. For every tested tree and depth, minimax and alpha-beta return the same value and a tie-equivalent action. The test varies utilities and successor order.

Solution 4.c - Alpha-beta and evaluation (1 points)

Question focus. Define an evaluation function as a weighted feature sum.

Answer. Features have stated perspective and scale, for example score difference, distance-to-objective, mobility, and threat. Positive weights favor MAX and terminal states bypass the heuristic.

Solution 4.d - Alpha-beta and evaluation (1 points)

Question focus. Measure the effect of ordering and evaluation features.

Answer. The report compares expanded nodes under good and bad ordering and performs at least one feature ablation tied to decision quality.

Solution 5.a - Integration and evidence (1 points)

Question focus. Run an end-to-end scenario that uses both pathfinding and adversarial choice.

Answer. The scenario makes the component handoff visible, validates every returned action, and records the chosen path and game decision.

Solution 5.b - Integration and evidence (1 points)

Question focus. Test terminal roots, no legal actions, repeated states, and a narrow pruning window.

Answer. Each test states expected output and detects a distinct semantic, mutation, cycle, or bound-update defect.

Solution 5.c - Integration and evidence (1 points)

Question focus. Report nodes expanded, prune count, decision value, and runtime.

Answer. Measurements use the same states, depth, hardware context, and tie order; conclusions distinguish correctness from speed.

Edge-case reasoning and expected behavior

- ▶ Return terminal utility and no action without calling the evaluator.
- ▶ Fail loudly or apply a documented pass rule; never return an arbitrary illegal action.
- ▶ Pathfinding deduplicates state keys; adversarial search avoids unsafe transposition reuse unless depth and bounds are encoded.
- ▶ Choose the first action in stable domain order.
- ▶ Pruning is safe when α is greater than or equal to β .

Grading rubric - part 1

Category	Points	Full-credit evidence
Architecture and pathfinding	4	Clean separation, deterministic behavior, and correct path costs.
A-star heuristic	3	Heuristic is safe, informative, and experimentally supported.
Minimax decision engine	5	Backups, player alternation, depth limits, and legal actions are correct.

Grading rubric - part 2

Category	Points	Full-credit evidence
Alpha-beta and evaluation	5	Bounds are updated correctly; evaluation features and evidence are sound.
Integration and evidence	3	End-to-end correctness, boundary cases, and technical interpretation.

Reflection exemplar

Answer. A strong response distinguishes pathfinding from game-state semantics, cites a minimal reproducer, identifies which contract was violated, and reports the regression test that now protects it.

Grading guidance. Award credit for a specific claim supported by technical evidence from the submitted work.